

Measuring What a Digital-Twin Validation Gate Buys: Counterfactual Labelling of Every Proposal Across Twin Fidelity

Draft for supervisor review, 23 September 2026
Twin-in-the-Loop project

Abstract—Placing a digital twin between an autonomous controller and a live system, and applying only the actions the twin predicts to be acceptable, is an established safeguard. Its value is usually reported as improvement over an ungated controller, which cannot separate the twin’s prediction from the effect of blocking. We present a controller-agnostic measurement method: every proposal, including rejected ones, receives a paired counterfactual label from the real simulator, and every twin-gated arm is compared with a reject-all gate that makes no prediction. We apply it to a deterministic edge-network simulator with a twin degraded along five fidelity axes, using 150 paired seeds, 6 fidelity levels, a rule-based and a scripted controller, and exact paired tests. Unvalidated action is harmful, and a low-fidelity twin is worse than no gate. The locally measured net benefit of the approved set crosses zero near fidelity 0.549 for the scripted controller; this crossing survives holding the twin’s queueing model constant. At the episode level the twin beats reject-all only for the rule controller at fidelity 1.0 and 0.8, by about 1.9% of baseline violations, carried by 14.7% of seeds. The ceiling on any gate’s gain is the benefit reject-all discards, which is small for both controllers. Classifier quality is best read where the harm threshold equals the gate tolerance. Local counterfactuals do not compose into episode effects. A 30-seed live-model study is reported descriptively only.

I. INTRODUCTION

Autonomous remediation needs a safeguard. Operating edge and IoT networks under faults requires remediation decisions faster than human operators can make them, and these decisions are increasingly delegated to automated controllers, including agents driven by large language models (LLMs) [1]–[3]. A wrong action applied to a live network can cause lasting damage.

The twin gate is established. A recurring design places a digital twin between the decision maker and the system: each proposed change is executed on the twin first and applied only if its predicted outcome is acceptable. Aether validates agent-proposed configuration changes on a network digital twin [4], process-plant work validates LLM-proposed actions in simulation and re-prompts on failure [5], and infrastructure-orchestration work describes a twin as a mandatory validation gate between an optimisation agent and the execution layer [6]. We adopt this architecture; we do not claim it as new.

What is missing is a measurement of what the twin buys. The consequence of a rejected action is rarely observed, so

evaluations compare a gated controller with an ungated one. That comparison conflates prediction with blocking: if most proposals are harmful, a gate that blocks everything looks excellent while predicting nothing. A twin is also never exact, so its value should be expected to depend on fidelity.

Contributions. (i) A controller-agnostic measurement method that labels every proposal, including rejected ones, with a paired counterfactual harm delta from the real system, and scores every twin-gated arm against a reject-all gate on the same seeds. (ii) A characterisation of how the twin’s value depends on fidelity along five controllable axes, including a control that holds the twin’s queueing model constant. (iii) Results at 150 paired seeds for two proposers, a live-model study as one further proposer, and a methodological finding that local counterfactuals do not compose into episode effects. LLM agents motivate the work; they are one tested proposer, not the evidence base.

II. RELATED WORK

Digital twins for change validation. Aether coordinates network-operations agents with a network digital twin to validate configuration changes on an ISP laboratory replica, with final approval by a human [4]. The process-plant study is the closest propose–simulate–validate–reprompt loop; its agent sees state and fault type and forces the action after retries are exhausted [5]. Orchestration work states the same gate-plus-retry pattern for compute infrastructure [6]. None sweeps twin fidelity or reports a reject-all control.

Remediation and network-agent benchmarks. MicroRemed [1], SLBench [7], NIKA [2] and NetAgentBench [3] measure the proposer and supply richer semantics than our simulator, but no counterfactual label for rejected actions.

Safety with utility. NetInjectBench shows that a static allowlist achieves safety at the cost of approved-change usefulness [8], and AgentDojo reports utility and attack success jointly [9]. Our reject-all arm applies the same principle in closed loop.

Predictive safety filters and shields. Simplex [10], predictive safety filters [11], model predictive shielding [12], [13] and model predictive control [14] forward-simulate a proposed action and fall back to a backup. These give guarantees

under model assumptions; our gate gives none. Rule-based shields score action-level safety labels as a classifier [15]; safe reinforcement learning is surveyed in [16]. We borrow the classifier view and add closed-loop outcomes and a no-prediction control.

Model fidelity. Lower-fidelity twins can detect failures comparably to higher-fidelity ones [17]; model-based reinforcement learning trades rollout length against model bias [18]; value-aware model learning weights model error by its effect on decisions [19].

Paired simulation and partial observability. Common random numbers reduce the variance of differences between simulated alternatives [20], [21]; our paired forks use this established technique and inherit its no-op continuation assumption. Diagnosability [22], [23] and planning under partial observability [24] motivate our fault-blind twin. Operational practice informs our action costs and symptom-only observation [25]–[28], and bounded agent loops [29], [30] motivate our retry cap.

III. METHOD

A. Simulator and fault injection

The ground truth is a deterministic, discrete-time simulator of an edge network with Poisson workloads, bounded service queues with overflow drops, and per-service SLO tracking. A `fork()` copies the full state including random-number streams, so branches replay identical exogenous randomness. Each seed fixes the workload and a fault schedule of 3 faults per episode with start ticks between 20 and 260 and durations of 20–80 ticks, drawn from CPU saturation, node crash, link degradation, link failure, memory leak and traffic surge. The gateway is not faultable in the primary protocol.

B. Observation, actions and costs

Controllers see symptoms only: summarised telemetry of latency, throughput, drops and utilisation, never the fault schedule or fault identities. They choose from a closed action set (`migrate`, `restart`, `scale`, `reroute`, `throttle`, `no_op`) validated by schema and semantic checks. Actions carry costs: restarts and migrations incur downtime and drop in-flight requests, so acting is not free.

C. Twin, fidelity axes and fault-blind rollout

At a decision tick with proposal a , the gate forks the live simulator, degrades the fork, and rolls two branches forward for $H = 20$ ticks, one with a and one with `no_op`. Both are *fault-blind*: faults active at the decision persist, future faults are hidden. The gate approves iff $\hat{V}_H(a) - \hat{V}_H(\text{no_op}) \leq \tau$, with $\tau = 0$. A rejection returns a reason; the controller may retry up to 2 times, then falls back to `no_op`. A scalar fidelity $f \in [0, 1]$ controls five axes: observation noise, state staleness, parameter drift, arrival-forecast error, and queueing simplification (exponential replaced by deterministic service), which switches on for $f < 0.5$. At $f = 1$ the twin is an exact fork that differs from reality only in not seeing future faults, so fidelity 1.0 is a harness sanity check rather than an achievable twin.

PLACEHOLDER: system architecture figure

Fault injector $\rightarrow$ simulator $\rightarrow$ telemetry $\rightarrow$ controller $\rightarrow$ validation $\rightarrow$ twin gate $\rightarrow$ executor; offline counterfactual labeller. Diagram to be inserted.

Fig. 1. System architecture (placeholder; no image file is available for this draft).

D. Counterfactual labelling

For every proposal, including rejected ones and retries, an offline labeller forks the *real* simulator with the fault schedule visible and rolls out the action followed by `no_op` and `no_op` throughout, each for H ticks with shared randomness. The harm delta is $D = V_H(a, \text{no_op}) - V_H(\text{no_op})$ in violation-ticks; $D < 0$ is beneficial, and an action is labelled harmful when $D > \theta$ with $\theta = 3$. The labeller never influences the live episode. From the labels we compute, per gate: $\text{benefit preserved} = \sum_{\text{approved}} \max(-D, 0) / \sum_{\text{all}} \max(-D, 0)$; $\text{harm prevented} = \sum_{\text{rejected}} \max(D, 0) / \sum_{\text{all}} \max(D, 0)$; and $\text{net } \Sigma D$, the per-seed mean of $\sum D$ over approved candidates, where negative means the approved set is locally better than rejecting everything.

E. Decision loop and architecture

Every 10 ticks the controller proposes, the proposal is validated, gated, and applied or rejected; episode outcome is total SLO violation-ticks over 120 ticks. Fig. 1 is a placeholder for the architecture diagram.

IV. EXPERIMENTAL SETUP

Seeds and conditions. 150 contiguous seeds, 6 fidelity levels (0.0 to 1.0 in steps of 0.2), and two queueing conditions: (i) the switch active as configured, and (ii) queueing simplification forced off at every level. This yields 3,150 episodes per condition and 6,300 in total. Condition (i) is primary unless stated.

Controllers and arms. A threshold *rule* controller (`migrate`, `restart` or `scale` on sustained violations) and a deterministic *scripted* scale-or-hold controller that stands in for an aggressive proposer and makes no model calls. Arms per controller: ungated; reject-all; random gating (approval probability 0.30 for rule and 0.747 for scripted); a schedule-aware gate that sees the fault schedule; and the twin gate at each fidelity. Reject-all equals the null controller on every seed.

Statistics. Episode comparisons are paired by seed: exact Wilcoxon signed-rank tests [31] computed by dynamic programming, Holm correction [32] within each controller’s 12 comparisons per condition, t -intervals, and seed-level percentile bootstrap intervals with 5,000 resamples sharing seeds across arms. Crossing fidelities are found by linear interpolation in each bootstrap resample.

Simulator validation. The simulator’s queue reproduces the M/M/1 mean queue length $L_q = \rho^2 / (1 - \rho)$ [33]: at $\rho = 0.6$ ($L_q = 0.90$) the measured mean over 360,000 ticks is within the test’s 15% tolerance. This covers one load regime, not all network behaviour.

V. RESULTS

A. Unvalidated action is harmful

For the scripted controller, ungated operation is worse than reject-all by +25.060 violation-ticks per episode, 95% CI [+22.320, +27.800], worse on 149 of 150 seeds (Holm $p = 3.4 \times 10^{-44}$). For the rule controller the ungated penalty is +3.393 [+1.338, +5.449] (Holm $p = 0.0011$). Acting without validation costs more than not acting (Table I).

B. A low-fidelity twin is worse than no gate

The scripted controller gated by twin@0.0 is worse than reject-all by +15.013 [+12.847, +17.179] (Holm $p = 1.9 \times 10^{-36}$; 126 seeds worse, 2 better). With queueing held constant the penalty is +12.433 [+10.550, +14.316]. Every scripted twin level up to 0.8 is significantly worse than reject-all after Holm correction (Fig. 3). A poor twin approves harmful actions that reject-all would have blocked.

C. The fidelity crossover in net ΣD

For the scripted controller, net ΣD falls monotonically from +3.033 at $f = 0$ to -0.620 at $f = 1$ and crosses zero at 0.555 [0.471, 0.677] (Fig. 2). The crossing lies between $f = 0.4$ and $f = 0.6$, exactly where the queueing switch flips, so as first observed it could not be attributed to fidelity. The crossing only became attributable after the control that holds the queueing model constant: it then sits at 0.549 [0.434, 0.677], monotone, with 4,997 of 5,000 resamples showing exactly one crossing. The 0.4→0.6 step in the held-constant condition is -1.113 [-1.753 , -0.507], while the switch itself is worth only -0.160 [-0.813 , $+0.480$] per seed at $f = 0.4$ ($p = 0.61$). The rule controller’s net ΣD is at or below zero at every level in both conditions (-0.147 to -2.120), with no crossing; held constant, its curve is flat between 0.4 and 0.6 ($+0.007$ [-0.133 , $+0.193$]) and is not fitted through.

D. Episode-level benefit for the rule controller

Only for the rule controller does the twin beat reject-all at the episode level: by -2.320 [-3.762 , -0.878] at fidelity 1.0 (exact Wilcoxon $p = 9.7 \times 10^{-4}$, Holm $p = 0.0078$) and -1.707 [-2.987 , -0.426] at 0.8 (Holm $p = 0.025$), identically in both queueing conditions. The effect is small: about 1.9% of the 122.1-tick reject-all baseline. It is concentrated in 22 of 150 seeds (14.7%), and 4 of those seeds went the wrong way (18 better). The fidelity-1.0 result survives Bonferroni correction across the 30 unique comparisons of both controllers and both conditions (adjusted $p = 0.029$); the fidelity-0.8 result does not (adjusted $p = 0.107$). The superseded 10-seed estimate for this comparison overstated the effect; all figures here are the 150-seed values. For the scripted controller, twin@1.0 is not distinguishable from reject-all ($+0.233$ [-0.156 , $+0.622$], $p = 0.287$, observed power 0.21).

E. Classifier quality

(a) Reading the classifier on the $\tau = \theta$ diagonal. At the operating setting $\tau = 0$, $\theta = 3$, the scripted twin@1.0 has FPR 0.256 [0.196, 0.323], but 56 of its 67 false positives (83.6%) are definitional: actions with $0 < D \leq 3$ that cause real harm below θ . On the diagonal that band is empty by construction, so every false positive is a genuine prediction error. At $\tau = \theta = 3$ recall is 0.996 and FPR is 0.065 [0.032, 0.105] (Table II, Fig. 4); the rule twin gives FPR 0.047–0.060 with recall 0.979–0.989. These rates rest on few negatives (262 scripted, 67 rule) and, for the rule controller, on 159 candidates in total; they describe a good filter on these candidates, not a general classifier.

(b) The operating point. The diagonal is where the classifier is read, not where the gate should run. At $\tau = 3$ the scripted gate’s net ΣD turns positive, $+0.173$ [-0.340 , $+0.600$], against -0.620 [-1.047 , -0.267] at $\tau = 0$, because a tolerance of 3 admits harmful scale-ups. The $\tau > 0$ rows relabel a fixed corpus of decisions taken at $\tau = 0$; they do not simulate a system running at $\tau > 0$.

F. The benefit ceiling

A gate cannot beat reject-all by more than the benefit reject-all discards. On the reject-all trajectory that benefit is 439 violation-ticks (2.93 per seed) in 23 of 150 seeds for the rule controller, and 52 ticks (0.35 per seed) in 12 seeds for the scripted controller, where reject-all’s trajectory holds 15 beneficial against 922 harmful scale-ups. Seeds with any beneficial candidate in any arm number 59 (rule) and 149 (scripted), but most of that benefit exists only on trajectories that earlier actions created. Harm prevented rises with fidelity for both controllers; benefit preserved for the rule controller is flat up to 0.6 and rises only at 0.8–1.0 (Fig. 5). The scripted null at fidelity 1.0 is therefore structural.

G. Live-model descriptive study

A hosted model (gemini-3.6-flash, temperature 0.0, one prompt) was run as a proposer on 30 seeds with ungated, reject-all, twin@1.0 and twin@0.6 arms. This study is descriptive. Valid non-no-op first proposals occurred at 51.9%–63.1% of scheduled decisions, and final parse failures at 14.7%–19.2%, each resolved to an explicit no-op fallback (Fig. 7). The model’s action mix differs from both baselines: on the ungated arm 39.6% scale, 31.7% restart and 28.6% migrate, against 81.4% scale for the rule controller and 100.0% for the scripted one (Fig. 6). Recoverable benefit on the reject-all trajectory is 252 ticks, 8.40 per seed in 12 of 30 seeds, larger per seed than for either baseline. The paired twin@1.0 – reject-all difference, -4.300 [-9.159 , $+0.559$] (exact Wilcoxon $p = 0.055$, Holm $p = 0.109$, observed power 0.48), is an underpowered pilot statistic; no episode-level effect is claimed.

H. Local counterfactuals do not compose into episode effects

Net ΣD is a sum of 20-tick counterfactuals with a no-op continuation, and it does not add up to closed-loop episode effects. For the scripted controller at fidelity 1.0, net ΣD

TABLE I
EPISODE-LEVEL PAIRED DIFFERENCES, TWIN — REJECT-ALL, VIOLATION-TICKS PER EPISODE, $n = 150$, CONDITION (I).

Controller	Arm	Mean	95% t -CI	Exact Wilcoxon p	Holm p	Seeds $\neq 0$ (better/worse)
Rule	ungated	+3.393	[+1.338, +5.449]	1.2×10^{-4}	0.0011	71 (14/57)
Rule	twin@0.0	+1.073	[−0.048, +2.194]	0.011	0.066	29 (6/23)
Rule	twin@0.8	−1.707	[−2.987, −0.426]	0.0036	0.025	18 (15/3)
Rule	twin@1.0	−2.320	[−3.762, −0.878]	9.7×10^{-4}	0.0078	22 (18/4)
Scripted	ungated	+25.060	[+22.320, +27.800]	2.8×10^{-45}	3.4×10^{-44}	149 (0/149)
Scripted	twin@0.0	+15.013	[+12.847, +17.179]	2.1×10^{-37}	1.9×10^{-36}	128 (2/126)
Scripted	twin@0.8	+0.560	[+0.090, +1.030]	0.0097	0.049	23 (6/17)
Scripted	twin@1.0	+0.233	[−0.156, +0.622]	0.287	0.574	17 (7/10)

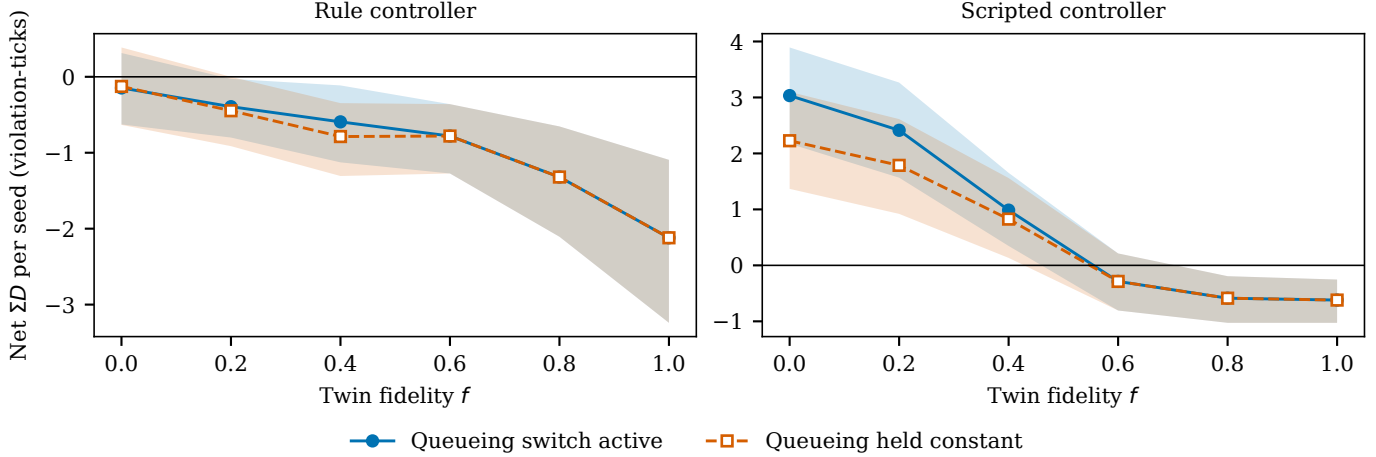


Fig. 2. Net ΣD per seed over approved candidates against twin fidelity, $n = 150$ seeds. Shaded bands are 95% seed-bootstrap intervals; solid circles: queueing switch active; dashed squares: queueing held constant. Negative values mean the approved set is locally better than reject-all. Scripted crossing: 0.555 [0.471, 0.677] (switch active), 0.549 [0.434, 0.677] (held constant).

TABLE II
TWIN@1.0 AS A CLASSIFIER OF HARMFUL ACTIONS ($D > \theta$). SCRIPTED: 1,167 CANDIDATES (29 BENEFICIAL); RULE: 159 (30 BENEFICIAL). FP SPLIT INTO DEFINITIONAL ($\tau < D \leq \theta$) AND PREDICTIVE.

Ctrl.	(τ, θ)	Recall	FPR	Neg. FP def.	FP pred.	
Scr.	(0,3)	0.996	0.256	262	56	11
Scr.	(1,1)	0.990	0.062	209	0	13
Scr.	(3,3)	0.996	0.065	262	0	17
Rule	(0,3)	0.989	0.075	67	3	2
Rule	(1,1)	0.979	0.047	64	0	3
Rule	(3,3)	0.989	0.060	67	0	4

is -0.620 per seed, predicting a twin advantage, while the episode difference is $+0.233$: the sign is wrong. The scripted net ΣD crossing near 0.555 has no episode-level counterpart; the episode curve approaches zero from above and never crosses. The mechanism is visible in single seeds of the earlier 10-seed ablation runs, cited here as worked examples and not as effect estimates: on seed 2, approved scale-ups summed to -13 locally, but the episode ended $+5$ ticks worse than reject-all, because approved scale-ups alter later states that the no-op continuation does not represent. For the rule controller the sign is right but migrations are underestimated: on seed 65 of the gateway-exposed ablation the observed episode effect was

$3.3\times$ the local sum with the gateway immune (-7 predicted, -23 observed) and $6.0\times$ with the gateway faultable (-8 predicted, -48 observed), because a migration’s downtime is paid inside the horizon while its benefit accrues for the rest of the fault. Local metrics describe the filter; performance claims need episode outcomes.

VI. THREATS TO VALIDITY

Shared simulator code. The twin and the labeller are forks of the same simulator, so model-structure error is represented only through the five degradation axes; fidelity 1.0 is a harness sanity check, not an attainable twin. **Relabelled corpus.** The $\tau > 0$ rows relabel decisions taken at $\tau = 0$. **Shared response cache.** In the live study one response cache served all arms, so arms reuse identical responses for identical prompts, and which arm first queries the model depends on execution order. **Synthetic only.** All results come from a synthetic simulator; no numerical comparison with systems evaluated on real replicas is meaningful. **Incomplete cost accounting.** The historical live study retained input and visible-output tokens but not thinking tokens or failed-request usage; its rate-card figure (\$10.53 for 5,644 billed calls) is not an invoice.

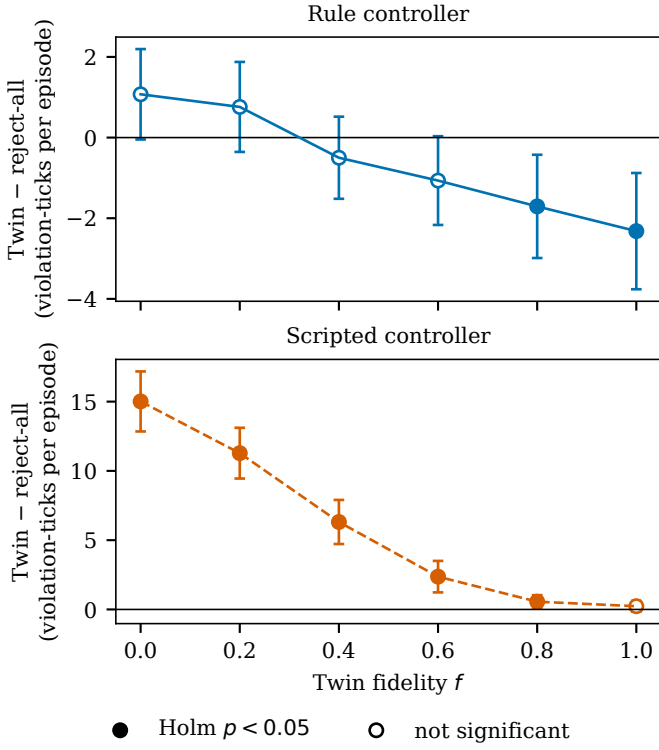


Fig. 3. Episode-level difference, twin – reject-all, by fidelity (condition (i), $n = 150$). Bars are 95% t -intervals; filled markers: Holm $p < 0.05$; open markers: not significant. Negative means the twin gate is better.

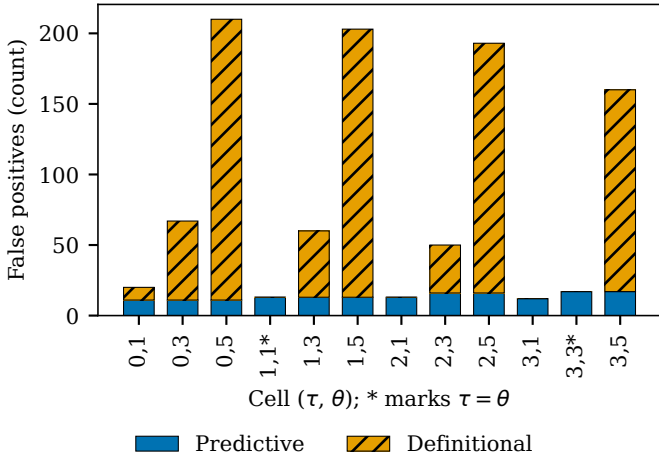


Fig. 4. False positives of the scripted controller’s twin@1.0 across (τ, θ) cells, split into definitional ($\tau < D \leq \theta$) and predictive. The definitional band is empty on the $\tau = \theta$ diagonal (marked *).

VII. LIMITATIONS AND FUTURE WORK

The live-model evidence is a single model, prompt and cache realisation at 30 seeds. The normal-approximation requirement computed from the 30-seed data is $n = 72$ seeds for 80% power on the twin@1.0 – reject-all comparison; a 72-seed live-model confirmation run is scheduled. A design with more restart and migration opportunities per episode would

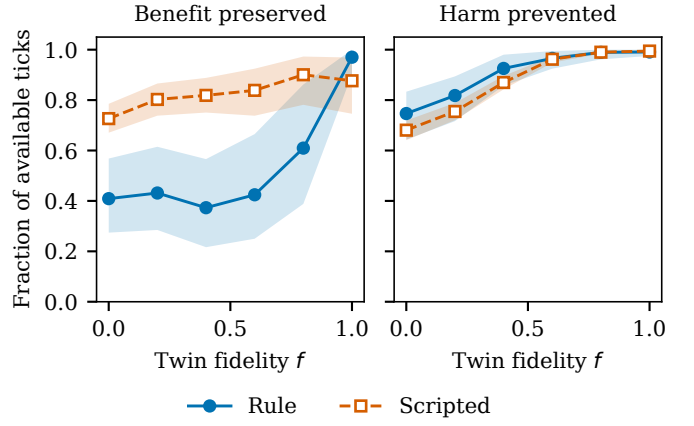


Fig. 5. Benefit preserved and harm prevented against twin fidelity (condition (i), $n = 150$), with 95% seed-bootstrap intervals.

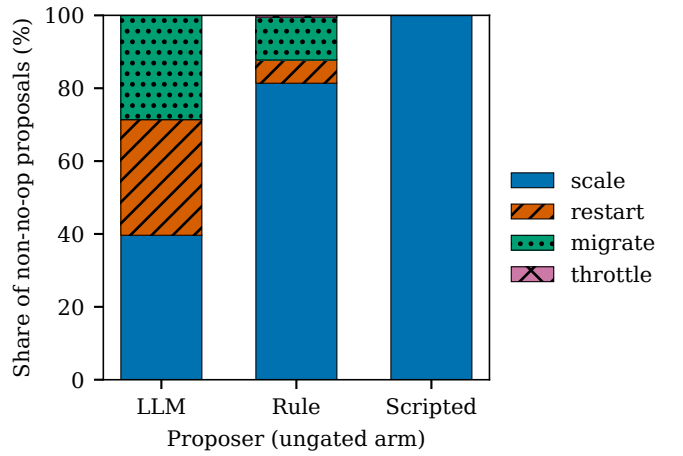


Fig. 6. Action-type mix of non-no-op proposals on the ungated arm: live model (30 seeds), rule and scripted controllers (150 seeds).

raise the benefit ceiling more efficiently than more seeds.

VIII. CONCLUSION

Comparing a twin gate with reject-all on counterfactually labelled proposals separates what the twin predicts from what it blocks. On 150 seeds the twin’s value depends strongly on fidelity: low fidelity is worse than no gate, and the net local benefit crosses zero near 0.549 once the queueing model is held constant. The episode-level gain is real for one controller, small, and concentrated in a few seeds, bounded by the benefit reject-all discards. Local counterfactual metrics are useful for describing the filter but do not compose into episode effects.

REFERENCES

- [1] “MicroRemed: Benchmarking LLMs in microservices remediation,” 2025, arXiv:2511.01166.
- [2] “A network arena for benchmarking AI agents on network troubleshooting,” 2025, arXiv:2512.16381.
- [3] “NetAgentBench: A state-centric benchmark for evaluating agentic network configuration,” 2026, arXiv:2604.09678.
- [4] “Aether: Network validation using agentic AI and digital twin,” 2026, arXiv:2604.18233.

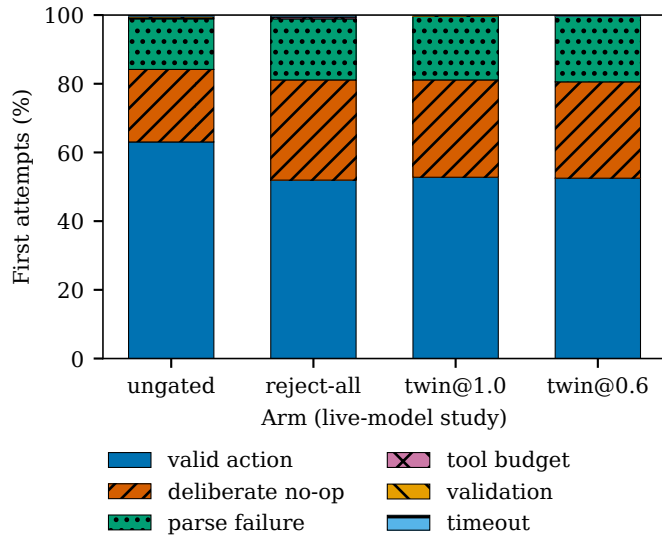


Fig. 7. Live-model first-attempt outcome at each scheduled decision, by arm (30 seeds, 360 first attempts per arm).

[5] “Leveraging LLM agents and digital twins for fault handling in process plants,” in *Proc. IEEE Int. Conf. Emerging Technologies and Factory Automation (ETFA)*, 2025, arXiv:2505.02076.

[6] “Digital-twin validation gate between an optimisation agent and the execution layer in infrastructure orchestration,” 2026, arXiv:2602.10900 and arXiv:2604.09705; bibliographic details to be verified.

[7] “SLBench,” 2026, arXiv:2607.09016.

[8] “NetInjectBench: Benchmarking indirect prompt injection in tool-using large language model agents for network operations,” 2026, arXiv:2607.10490.

[9] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents,” in *Proc. NeurIPS Datasets and Benchmarks Track*, 2024.

[10] M. Rivera *et al.*, “An architectural description of the Simplex architecture,” Software Engineering Institute, Carnegie Mellon University, Tech. Rep. CMU/SEI-96-TR-006, 1996.

[11] K. P. Wabersich and M. N. Zeilinger, “A predictive safety filter for learning-based control of constrained nonlinear dynamical systems,” *Automatica*, vol. 129, p. 109597, 2021.

[12] O. Bastani, “Safe reinforcement learning with nonlinear dynamics via model predictive shielding,” 2019, arXiv:1905.10691.

[13] S. Li and O. Bastani, “Robust model predictive shielding for safe reinforcement learning with stochastic dynamics,” 2019, arXiv:1910.10885.

[14] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed. Madison, WI, USA: Nob Hill Publishing, 2017.

[15] Shperberg *et al.*, “A rule-based shield: Accumulating safety rules from catastrophic action effects,” in *Proc. Conference on Lifelong Learning Agents (CoLLAs)*, ser. PMLR, vol. 199, 2022.

[16] “A survey of safe reinforcement learning and constrained MDPs: A technical survey on single-agent and multi-agent safety,” 2025, arXiv:2505.17342.

[17] C. Su, B. Hicks, and A. Nassehi, “Investigating the influence of fidelity on the capability of a digital twin to detect material extrusion failures,” *Journal of Intelligent Manufacturing*, vol. 35, pp. 2263–2276, 2024.

[18] M. Janner, J. Fu, M. Zhang, and S. Levine, “When to trust your model: Model-based policy optimization,” in *Proc. NeurIPS*, 2019.

[19] A.-m. Farahmand, “Iterative value-aware model learning,” in *Proc. NeurIPS*, 2018.

[20] W.-N. Yang and B. L. Nelson, “Using common random numbers and control variates in multiple-comparison procedures,” *Operations Research*, vol. 39, no. 4, pp. 583–591, 1991.

[21] Yadav *et al.*, “Using common random numbers for simulation-based

planning with rollouts,” in *Proc. Reinforcement Learning Conference (RLC)*, 2026.

[22] M. Sampath, R. Sengupta, S. Lafortune, K. Sinnamohideen, and D. Teneketzis, “Diagnosability of discrete-event systems,” *IEEE Transactions on Automatic Control*, vol. 40, no. 9, pp. 1555–1575, 1995.

[23] M. P. Cabasino, A. Giua, and C. Seatzu, “Diagnosability of discrete-event systems using labeled Petri nets,” *IEEE Transactions on Automation Science and Engineering*, 2014.

[24] D. Silver and J. Veness, “Monte-Carlo planning in large POMDPs,” in *Proc. NeurIPS*, 2010.

[25] The Kubernetes Authors, “Controllers,” Kubernetes documentation, <https://kubernetes.io/docs/concepts/architecture/controller/>.

[26] —, “Horizontal pod autoscaling,” Kubernetes documentation, <https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/>.

[27] “Addressing cascading failures,” in *Site Reliability Engineering: How Google Runs Production Systems*, B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds. O’Reilly Media, 2016.

[28] “Monitoring distributed systems,” in *Site Reliability Engineering: How Google Runs Production Systems*, B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds. O’Reilly Media, 2016.

[29] Liu *et al.*, “Budget-aware tool use enables effective agent scaling,” 2025, arXiv:2511.17006.

[30] LangChain, “Graph API: Recursion limit,” LangGraph documentation, <https://docs.langchain.com/oss/python/langgraph/graph-api>.

[31] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[32] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[33] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. New York, NY, USA: Wiley, 1975.